Vegas.lean

```lean
import Mathlib.Data.List.Basic
import Mathlib.Data.Rat.Init

abbrev Player : Type := Nat
abbrev YieldId : Type := Nat

inductive Ty where
  | int : Ty
  | bool : Ty
deriving Repr, DecidableEq

abbrev Val : Ty → Type
  | .int => Int
  | .bool => Bool

abbrev Ctx : Type := List Ty

inductive Var : Ctx → Ty → Type where
  | vz {Γ τ} : Var (τ :: Γ) τ
  | vs {Γ τ τ'} (x : Var Γ τ) : Var (τ' :: Γ) τ
deriving Repr

def Var.weaken {Γ : Ctx} {τ τ' : Ty} : Var Γ τ → Var (τ' :: Γ) τ
  | .vz => .vs .vz
  | .vs x => .vs (Var.weaken x)

def Env : Ctx → Type
  | [] => Unit
  | τ :: Γ => Val τ × Env Γ

def Env.get {Γ : Ctx} {τ : Ty} (ρ : Env Γ) (x : Var Γ τ) : Val τ :=
  match x, ρ with
  | .vz, (v, _) => v
  | .vs v, (_, ρ) => get ρ v

inductive Expr : Ctx → Ty → Type where
  | var {Γ τ} (x : Var Γ τ) : Expr Γ τ
  | constInt {Γ} (i : Int) : Expr Γ .int
  | constBool {Γ} (b : Bool) : Expr Γ .bool
  | addInt {Γ} (l r : Expr Γ .int) : Expr Γ .int
  | eqInt {Γ} (l r : Expr Γ .int) : Expr Γ .bool
  | andBool {Γ} (l r : Expr Γ .bool) : Expr Γ .bool
  | notBool {Γ} (b : Expr Γ .bool) : Expr Γ .bool
deriving Repr

def evalExpr {Γ : Ctx} {τ : Ty} : Expr Γ τ → Env Γ → Val τ
  | .var x, env => env.get x
  | .constInt i, _ => i
  | .constBool b, _ => b
```

```
    | .addInt l r, env => evalExpr l env + evalExpr r env
    | .eqInt l r, env => decide (evalExpr l env = evalExpr r env)
    | .andBool l r, env => evalExpr l env && evalExpr r env
    | .notBool b, env => !(evalExpr b env)

def weakenExpr {Γ : Ctx} {τ τ' : Ty} : Expr Γ τ → Expr (τ' :: Γ) τ
    | .var x => .var (Var.weaken x)
    | .constInt i => .constInt i
    | .constBool b => .constBool b
    | .addInt l r => .addInt (weakenExpr l) (weakenExpr r)
    | .eqInt l r => .eqInt (weakenExpr l) (weakenExpr r)
    | .andBool l r => .andBool (weakenExpr l) (weakenExpr r)
    | .notBool b => .notBool (weakenExpr b)

variable {α β : Type}

structure WDist (α : Type) where
  weights : List (α × Rat)
deriving Inhabited
def WDist.pure (x : α) : WDist α := [(x, 1)]
def WDist.zero : WDist α := []
def WDist.bind (d : WDist α) (f : α → WDist β) : WDist β :=
    d.weights.flatMap (fun (a, w) => (f a).weights.map (fun (b, w') => (b, w * w')))
def WDist.map (f : α → β) (d : WDist α) : WDist β :=
    d.weights.map (fun (a, w) => (f a, w))

structure View (Γ : Ctx) where
  Δ : Ctx
  proj : Env Γ → Env Δ

abbrev Act {Γ : Ctx} (v : View Γ) (τ : Ty) : Type :=
  Env v.Δ → List (Val τ)

abbrev ObsKernel {Γ : Ctx} (v : View Γ) (τ : Ty) : Type :=
  Env v.Δ → WDist (Val τ)

structure GameEvent where
  name : String
  payoff : Player → Rat

inductive VegasProg : Ctx → Ty → Type where
  | ret {Γ τ} (e : Expr Γ τ) : VegasProg Γ τ
  | letDet {Γ τ τ'} (e : Expr Γ τ') (k : VegasProg (τ' :: Γ) τ) : VegasProg Γ τ
  | sample {Γ τ τ'} (id : YieldId) (v : View Γ) (K : ObsKernel v τ')
      (k : VegasProg (τ' :: Γ) τ) : VegasProg Γ τ
  | choose {Γ τ τ'} (id : YieldId) (who : Player) (v : View Γ) (A : Act v τ')
      (k : VegasProg (τ' :: Γ) τ) : VegasProg Γ τ
  | observe {Γ τ} (cond : Expr Γ .bool) (k : VegasProg Γ τ) : VegasProg Γ τ
  | emit {Γ τ} (name : Env Γ → String) (payoff : Env Γ → Player → Rat)
      (k : VegasProg Γ τ) : VegasProg Γ τ

/- Optional resolver for partially-specified profiles. -/
structure PProfile where
```

```
    choose? :
      {Γ : Ctx} → {τ : Ty} →
      (who : Player) → (id : YieldId) → (v : View Γ) → (A : Act v τ) →
      Option (Env v.Δ → WDist (Val τ))

/-- A partial profile is complete when every decision site is resolved. -/
def PProfile.Complete (π : PProfile) : Prop :=
      {Γ : Ctx} {τ : Ty}
      (who : Player) (id : YieldId) (v : View Γ) (A : Act v τ),
      (π.choose? who id v A).isSome

/-- Extract the resolver function from a complete partial profile. -/
def PProfile.chooseOfComplete
      (π : PProfile) (hπ : π.Complete)
      {Γ : Ctx} {τ : Ty}
      (who : Player) (id : YieldId) (v : View Γ) (A : Act v τ) :
      Env v.Δ → WDist (Val τ) :=
    Option.get (π.choose? who id v A) (hπ who id v A)

abbrev Trace : Type := List GameEvent

/-- Evaluate a Vegas protocol game under a profile. -/
def evalVegas (π : PProfile) (hπ : π.Complete) {Γ : Ctx} {τ : Ty} :
      VegasProg Γ τ → Env Γ → WDist (Val τ × Trace)
  | .ret e, env =>
      WDist.pure (evalExpr e env, [])
  | .letDet e k, env =>
      evalVegas π hπ k (evalExpr e env, env)
  | .sample _id v K k, env =>
      WDist.bind (K (v.proj env)) (fun x => evalVegas π hπ k (x, env))
  | .choose id who v A k, env =>
      WDist.bind (π.chooseOfComplete hπ who id v A (v.proj env))
        (fun x => evalVegas π hπ k (x, env))
  | .observe cond k, env =>
      if evalExpr cond env then evalVegas π hπ k env else WDist.zero
  | .emit name payoff k, env =>
      WDist.map (fun (v, tr) => (v, name env , payoff env  :: tr)) (evalVegas π hπ k env)

def applyProfile (π : PProfile) {Γ : Ctx} {τ : Ty} : VegasProg Γ τ → VegasProg Γ τ
  | .ret e => .ret e
  | .letDet e k => .letDet e (applyProfile π k)
  | .sample id v K k => .sample id v K (applyProfile π k)
  | .choose id who v A k =>
      match π.choose? who id v A with
      | some Kdec => .sample id v Kdec (applyProfile π k)
      | none => .choose id who v A (applyProfile π k)
  | .observe cond k => .observe cond (applyProfile π k)
  | .emit name payoff k => .emit name payoff (applyProfile π k)

def NoChoose {Γ : Ctx} {τ : Ty} : VegasProg Γ τ → Prop
  | .ret _ => True
  | .letDet _ k => NoChoose k
  | .sample _ _ _ k => NoChoose k
```

```
    | .choose _ _ _ _ _ => False
    | .observe _ k => NoChoose k
    | .emit _ _ k => NoChoose k

  def tracePayoff (tr : Trace) (who : Player) : Rat :=
    tr.foldr (fun ev acc => acc + ev.payoff who) 0

  def EU_game {τ : Ty} (d : WDist (Val τ × Trace)) (who : Player) : Rat :=
    d.weights.foldr
      (fun (xw : (Val τ × Trace) × Rat) acc =>
        acc + xw.2 * tracePayoff xw.1.2 who)
      0

  def EU {Γ : Ctx} {τ : Ty}
      (p : VegasProg Γ τ) (π : PProfile) (hπ : π.Complete)
      (env : Env Γ) (who : Player) : Rat :=
    EU_game (evalVegas π hπ p env) who

```